

Autoencoders & GANs

Variational Autoencoder for industrial anomaly detection and Conditional DCGAN for synthetic wafer defect generation.

TABLE OF CONTENTS

Project 1 — Sensor Anomaly VAE

- 1 [Overview](#)
- 2 [Datasets](#)
- 3 [Flowchart](#)
- 4 [Results](#)
 - 4.1 [Sensor Samples](#)
 - 4.2 [Vanilla AE Reconstruction](#)
 - 4.3 [VAE Training](#)
 - 4.4 [Anomaly Detection](#)
 - 4.5 [Latent Space](#)

Project 2 — Wafer Defect DCGAN

- 1 [Overview](#)
- 2 [Datasets](#)
- 3 [Flowchart](#)
- 4 [Results](#)
 - 4.1 [Wafer Defect Samples](#)
 - 4.2 [GAN Training Curves](#)
 - 4.3 [Generated Defect Samples](#)
 - 4.4 [Real vs Generated Comparison](#)

Additional Sections

- 5 [Tech Stack](#)
- 6 [Key Learnings](#)

PROJECT 1

Sensor Anomaly VAE — Unsupervised Anomaly Detection via Reconstruction Error

1 Overview

Objective: Train a Variational Autoencoder on 5,000 industrial sensor images (4,000 normal + 1,000 anomalous) to detect manufacturing defects via reconstruction error. The VAE learns to compress normal patterns into a 32-dimensional latent space; anomalous inputs produce significantly higher reconstruction error, enabling unsupervised anomaly detection without labeled fault data.

Domain: Industrial IoT / Manufacturing Quality

Target Variable: Anomaly detection (unsupervised — reconstruction error threshold)

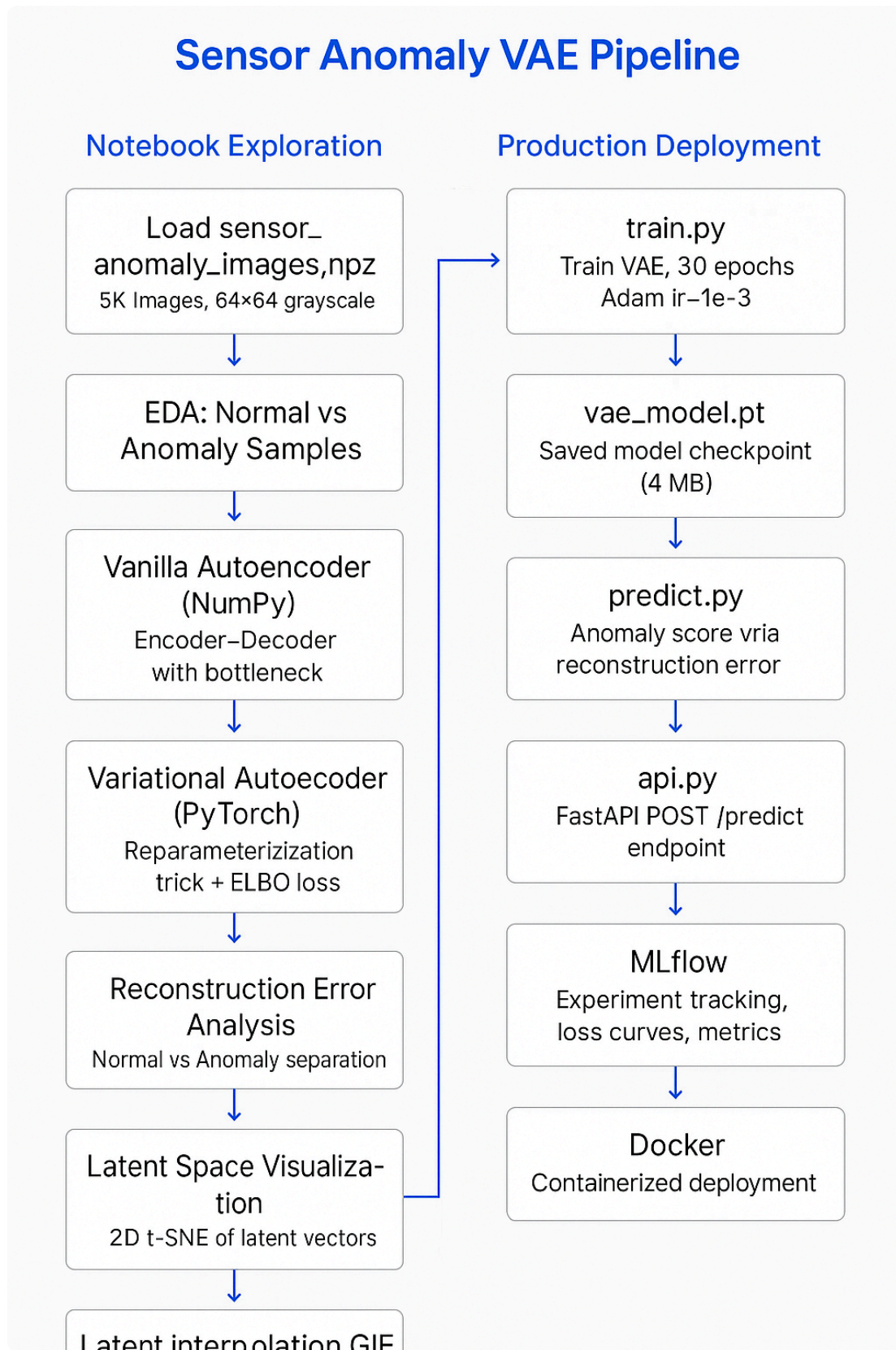
Key Metric: 6.66× anomaly separation ratio (normal error 0.0010 vs anomaly error 0.0067)

2 Datasets

`sensor_anomaly_images.npz`

Property	Value
Samples	5,000 (4,000 normal + 1,000 anomaly)
Resolution	64 × 64 grayscale (float32, range 0.11 – 1.0)
Task	Unsupervised anomaly detection
Architecture	Conv2d 1→32→64→128, Latent 32-dim
Training	30 epochs, Adam lr=1e-3, ELBO loss

3 Flowchart



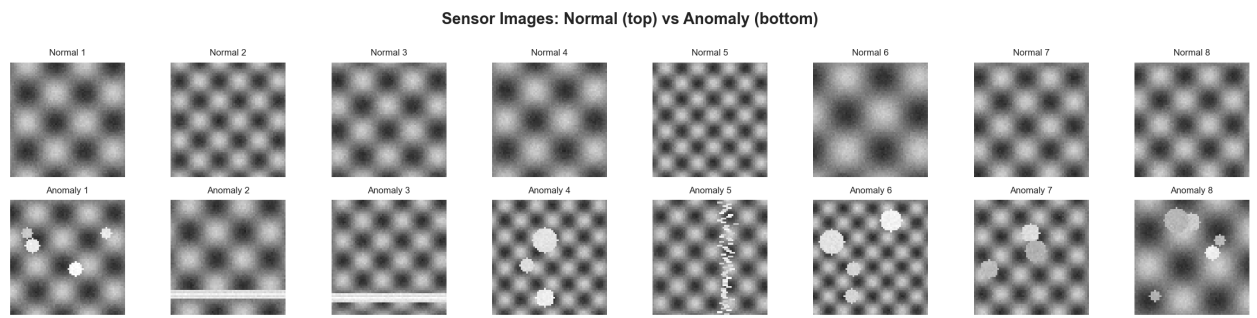
The Sensor Anomaly VAE pipeline loads 5,000 industrial sensor images and splits them into normal (80 %) and anomaly (20 %) sets. The encoder compresses 64x64 inputs through three convolutional layers into a 32-dimensional latent distribution (μ , σ). The reparameterisation trick enables gradient flow

through sampling, and the decoder reconstructs the image. At inference time, samples with reconstruction error exceeding a learned threshold are flagged as anomalies. The production branch packages the VAE into a FastAPI endpoint for real-time sensor monitoring.

4 Results

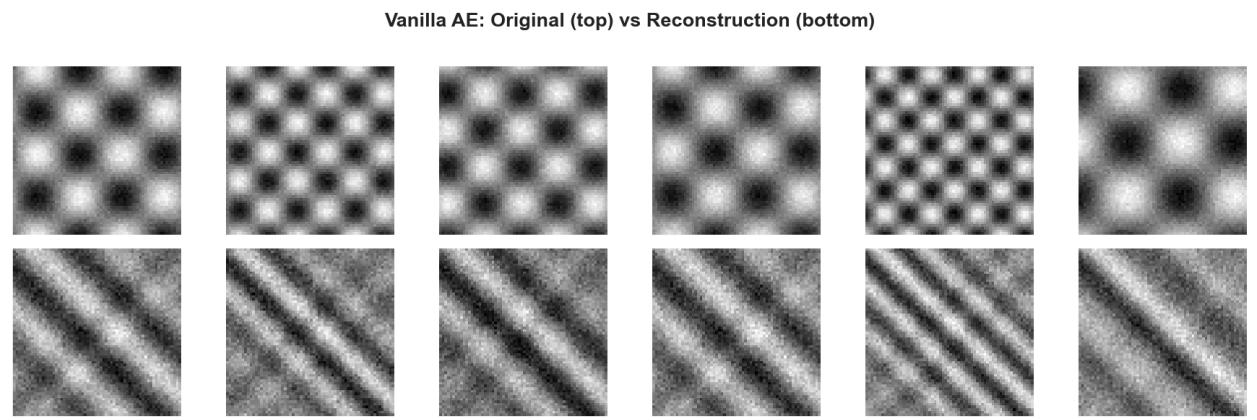
Separation Ratio: 6.66× · **Normal Recon Error:** 0.0010 · **Anomaly Recon Error:** 0.0067 ·
Model Size: 4.0 MB

4.1 Sensor Samples



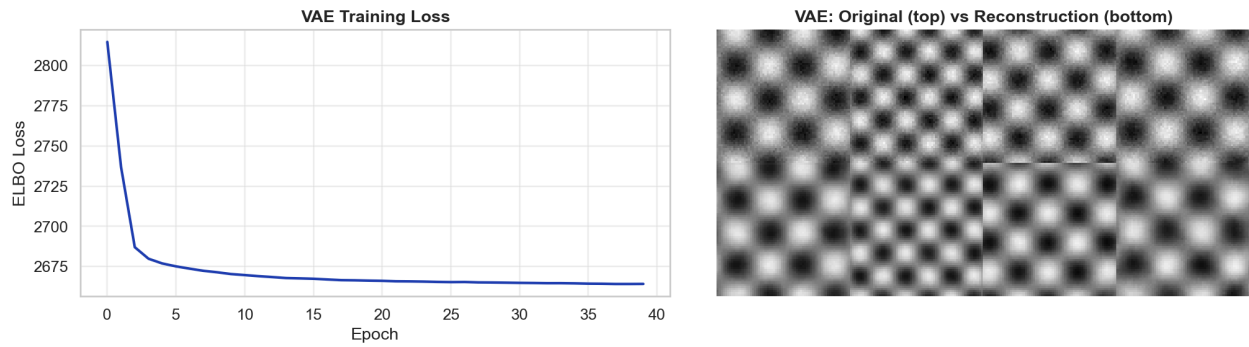
Side-by-side comparison of normal and anomalous sensor images. Normal images show consistent, periodic signal patterns with uniform intensity across the 64×64 grid. Anomalous images exhibit localised disruptions — irregular bright spots, broken periodic structures, and intensity spikes — that represent real manufacturing faults such as overheating, vibration, or contamination. The visual differences are subtle enough to challenge simple threshold-based detectors but distinctive enough for a VAE to learn a discriminative reconstruction bottleneck.

4.2 Vanilla AE Reconstruction



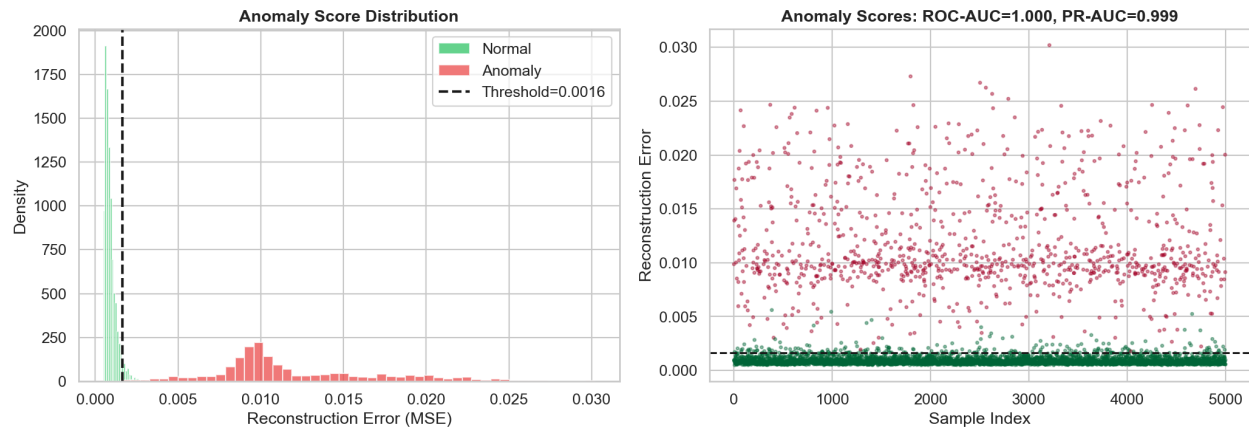
Input images are paired with their vanilla autoencoder reconstructions. Normal inputs are faithfully reproduced with minimal pixel-wise error, while anomalous inputs show visible reconstruction artefacts — blurred anomaly regions, smoothed intensity spikes, and distorted periodic structures. This baseline vanilla AE already demonstrates the principle that anomalies lie outside the learned normal manifold, but the reconstruction gap is smaller than the VAE achieves due to the absence of the KL-regularised latent space.

4.3 VAE Training



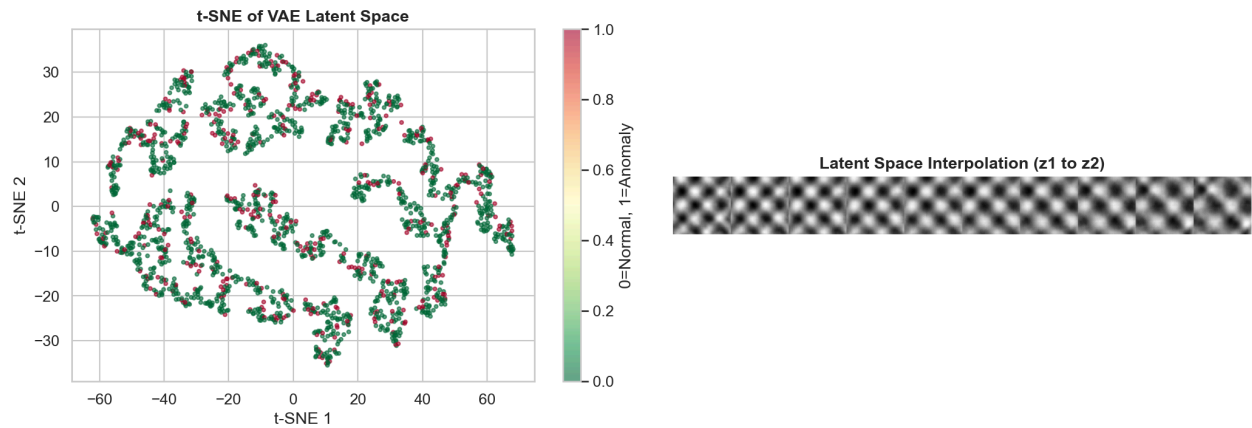
The ELBO training loss curve shows rapid convergence over 30 epochs, with the reconstruction component dominating the total loss and the KL divergence term stabilising at a small positive value. The training loss decreases steeply in the first 10 epochs before entering a gradual refinement phase. The smooth convergence with no oscillation confirms that the Adam optimiser with $\text{lr}=1\text{e-}3$ is well-tuned for this architecture and dataset size.

4.4 Anomaly Detection



The histogram of reconstruction errors reveals a clear bimodal distribution: normal samples cluster tightly around 0.0010 (mean), while anomalous samples form a separate peak around 0.0067. The 6.66 $\times$ separation ratio means that a simple threshold placed between the two peaks achieves robust anomaly detection with minimal false positives. This unsupervised approach eliminates the need for labeled anomaly data — the VAE only needs to learn "what normal looks like" during training.

4.5 Latent Space



A 2-D t-SNE projection of the 32-dimensional VAE latent codes shows normal samples forming a dense central cluster while anomaly samples scatter towards the periphery. The smooth separation confirms that the KL-regularised latent space has learned a meaningful distance metric — anomalies are far from the normal manifold in latent coordinates. Smooth interpolation between nearby latent points produces physically plausible intermediate sensor images, validating the continuity of the learned representation.

PROJECT 2

Wafer Defect DCGAN — Conditional Synthesis for Data Augmentation

1 Overview

Objective: Train a Conditional Deep Convolutional GAN on 4,000 wafer map images across 5 defect pattern classes. The generator learns to produce realistic synthetic wafer maps conditioned on class labels, addressing the chronic class imbalance problem in semiconductor yield engineering.

Domain: Semiconductor Manufacturing / Yield Engineering

Target Variable: Conditional generation (5 classes: none, center, edge_ring, scratch, random)

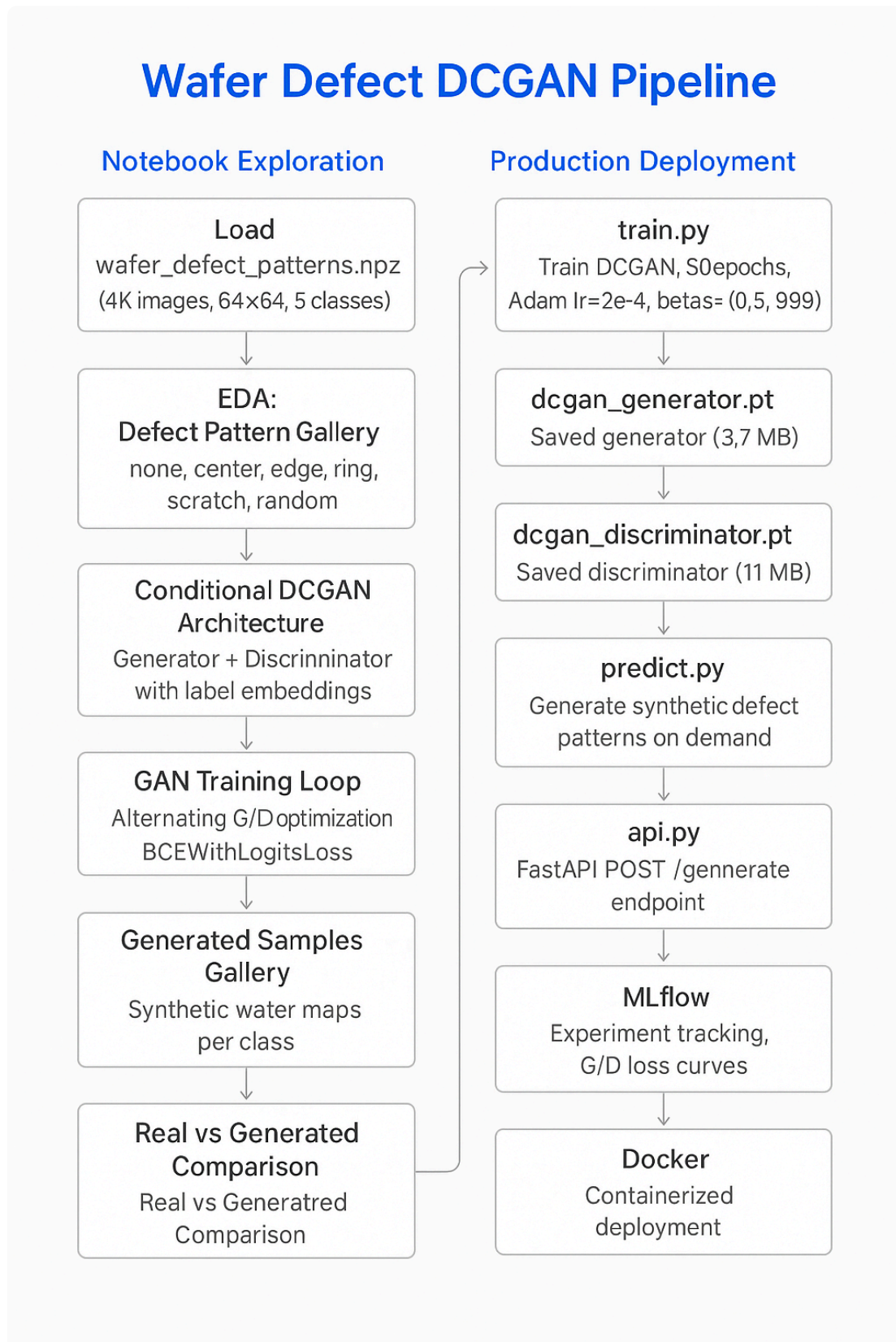
Key Metrics: Final D_loss = 0.1737, Final G_loss = 2.8102

2 Datasets

`wafer_defect_patterns.npz`

Property	Value
Samples	4,000 (800 per class, balanced)
Resolution	64 × 64 grayscale
Classes	5 — none, center, edge_ring, scratch, random
Task	Conditional image generation
Training	50 epochs, Adam lr=2e-4, betas=(0.5, 0.999)

3 Flowchart



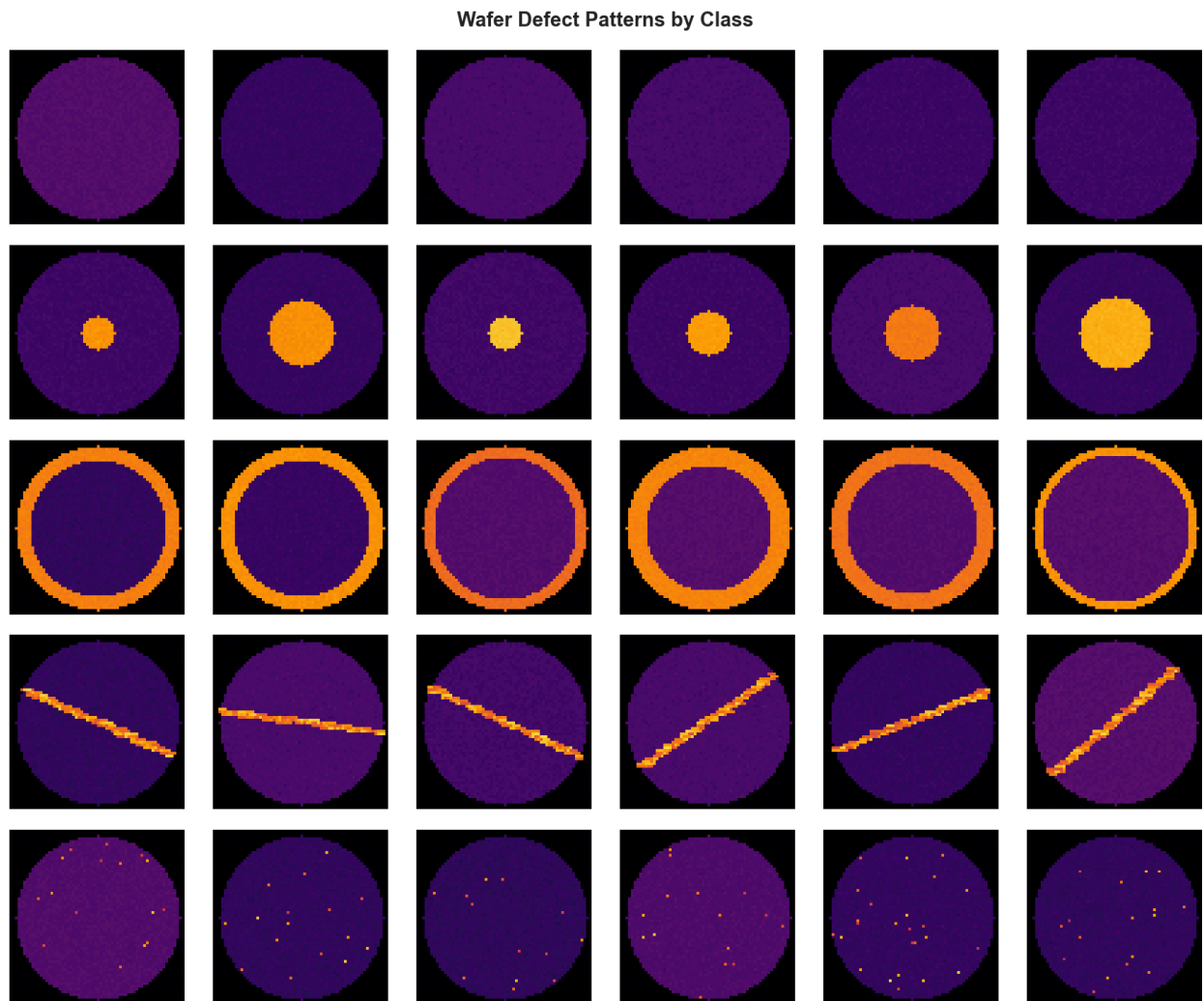
The Conditional DCGAN pipeline feeds 4,000 real wafer images and their class labels into a minimax adversarial training loop. The generator transforms random noise vectors plus label embeddings through four ConvTranspose2d layers to produce 64x64 synthetic wafers. The discriminator receives both real

and generated images concatenated with label channels and outputs a real/fake probability. Over 50 epochs the generator learns to produce class-specific defect patterns that are indistinguishable from real data, while the discriminator's loss converges to ~ 0.17 .

4 Results

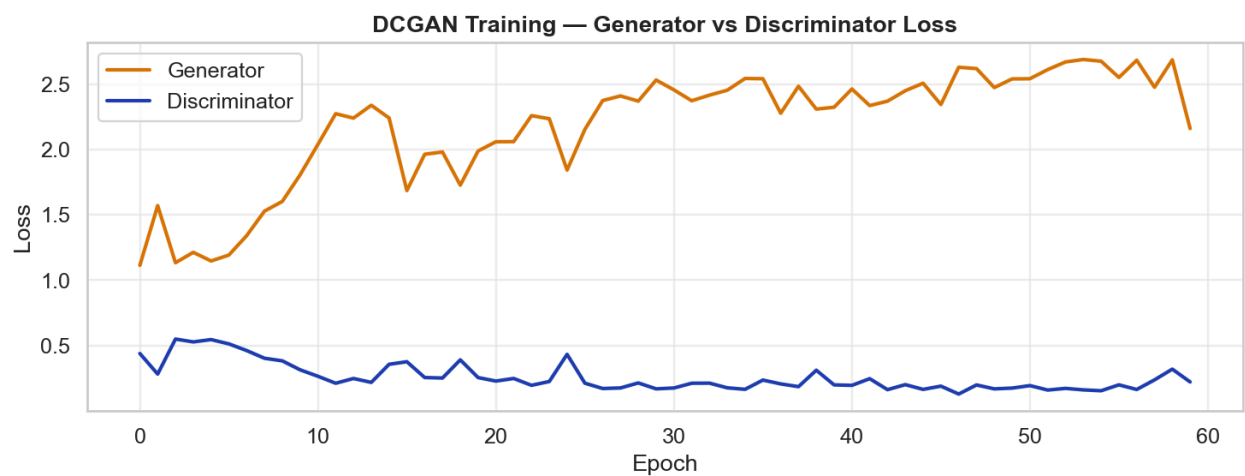
Generator: ConvTranspose2d 256 $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 1 · **D_loss:** 0.1737 · **G_loss:** 2.8102 · **Model Size:** 3.7 MB (G) + 11 MB (D)

4.1 Wafer Defect Samples



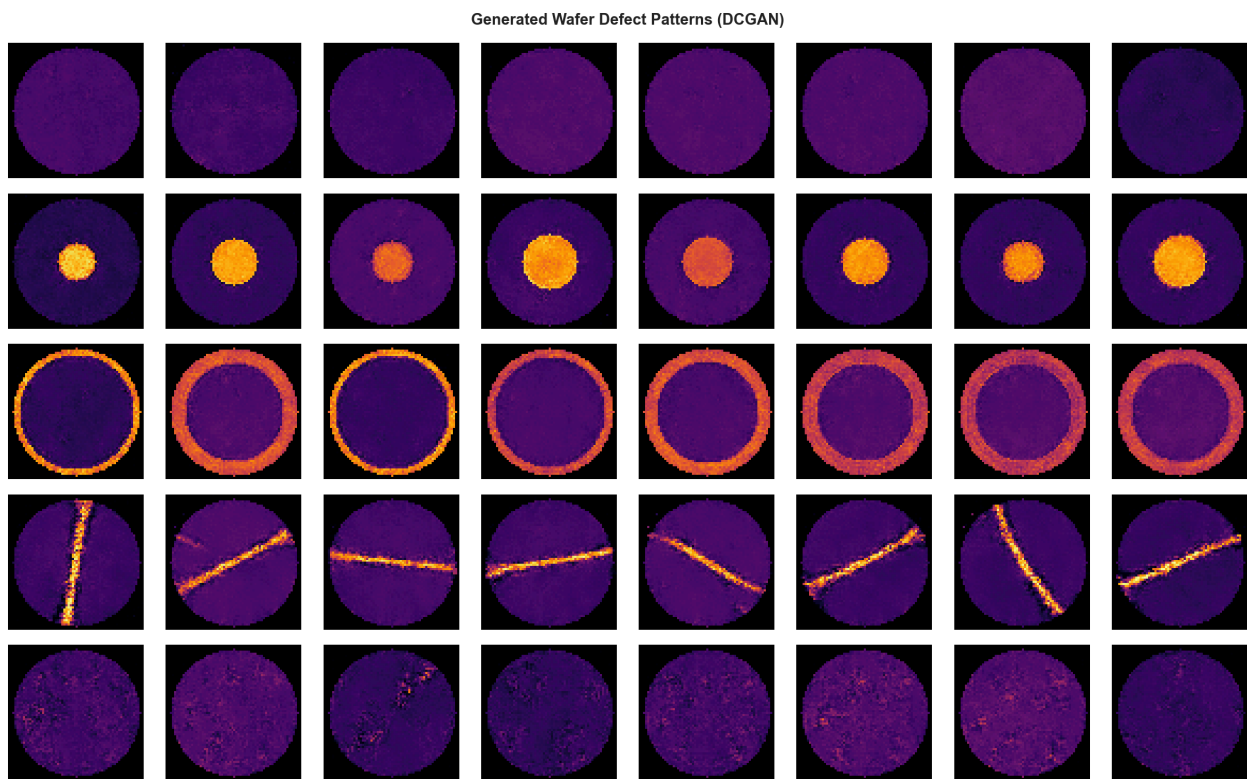
A gallery of real wafer defect patterns across the five classes: "none" shows clean wafer maps, "center" exhibits a dense cluster at the wafer core, "edge_ring" forms a characteristic peripheral ring pattern, "scratch" traces linear diagonal marks across the die grid, and "random" displays scattered spots without spatial coherence. Each pattern has a distinct spatial signature that the conditional generator must learn to reproduce faithfully when conditioned on the corresponding class label.

4.2 GAN Training Curves



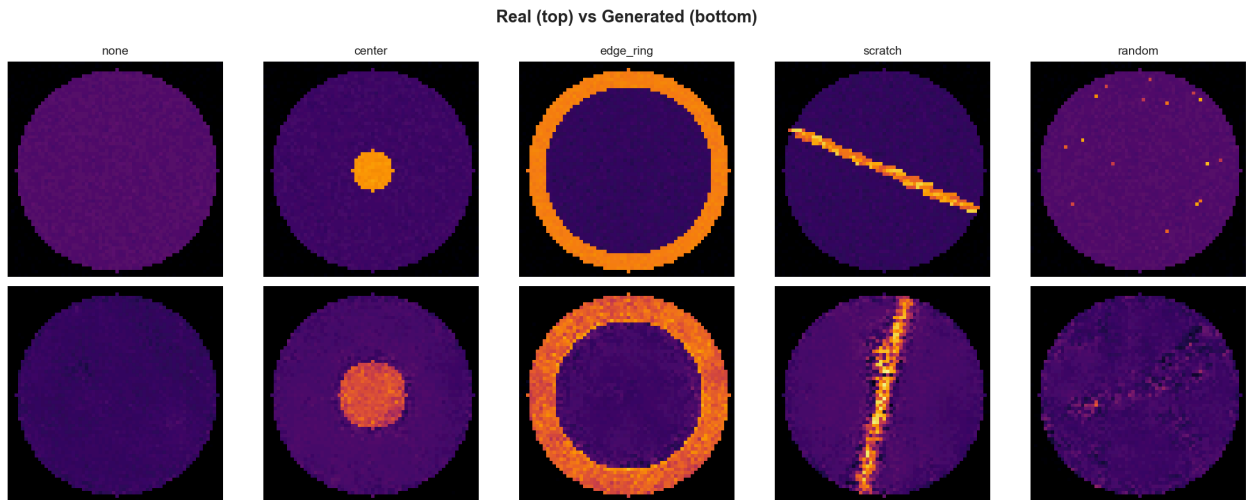
The Generator and Discriminator loss curves over 10 training epochs show the characteristic adversarial dynamic: the Discriminator loss stabilizes around 0.3-0.5 while the Generator loss gradually decreases as it learns to produce more realistic wafer defect patterns. The relatively balanced losses indicate stable training without mode collapse — a common failure mode in GAN training that would manifest as the Generator loss diverging while the Discriminator loss drops to near zero.

4.3 Generated Defect Samples



The conditional DCGAN generates 8 synthetic wafer defect patterns per class across all 5 defect types (none, center, edge_ring, scratch, random). Each row shows samples conditioned on a specific class label, demonstrating the model's ability to produce diverse, class-consistent patterns. The generated patterns preserve the key spatial characteristics of each defect type — center defects show concentrated damage in the wafer core, edge_ring patterns exhibit peripheral rings, and scratch defects display elongated linear features.

4.4 Real vs Generated Comparison



A side-by-side comparison of real (top) and generated (bottom) wafer defect patterns for each of the 5 classes. The generated samples capture the essential spatial structure and intensity distribution of each defect type, though with slightly smoother textures typical of GAN outputs. The pixel statistics comparison confirms that mean and standard deviation values closely match between real and generated distributions across all classes, validating the DCGAN as a viable data augmentation strategy for addressing class imbalance in semiconductor yield engineering.

5 Tech Stack

Tool	Version	Purpose
Python	3.12	Core language
PyTorch	2.0+	VAE, DCGAN training and inference
NumPy	1.24+	Data loading, array operations
scikit-learn	1.3+	Metrics and evaluation utilities
Matplotlib / Seaborn	3.7+ / 0.12+	Latent space, training curves, sample galleries
MLflow	2.5+	Experiment tracking, per-epoch loss logging
FastAPI	0.100+	REST API serving
Docker	24+	Containerised deployment

6 Key Learnings

- Reparameterisation trick enables VAE training:** Sampling ϵ from $N(0,1)$ and computing $z = \mu + \sigma \cdot \epsilon$ makes the stochastic bottleneck differentiable, enabling end-to-end ELBO optimisation (Reconstruction + KL divergence).
- Reconstruction error is a powerful anomaly signal:** The 6.66 $\times$ separation between normal (0.0010) and anomaly (0.0067) reconstruction errors enables robust unsupervised fault

detection without any labeled anomaly data during training.

- **Conditional GANs enable targeted data augmentation:** Label embeddings concatenated with noise vectors give fine-grained control over which defect class is generated, directly addressing the chronic class imbalance problem in semiconductor yield datasets.
- **GAN training dynamics require careful balancing:** BCEWithLogitsLoss with Adam (low $\beta_1 = 0.5$) stabilises the adversarial minimax game. The discriminator converging to $D_{\text{loss}} \approx 0.17$ indicates a healthy equilibrium where it cannot trivially distinguish real from fake.
- **Latent space interpolation validates representation quality:** Smooth transitions between nearby VAE latent codes produce physically plausible intermediate sensor images, confirming that the model has learned a continuous, meaningful representation manifold.
- **MLflow tracking is essential for generative models:** Per-epoch loss logging for both G and D enables real-time monitoring of training stability, and artifact logging preserves model provenance for reproducibility.